

Infrastructure and Containerization

Module 3 Assignment 2

1 Single-stage vs. multi-stage Docker

The API is a small FastAPI service backed by a TF-IDF + MultinomialNB pipeline. The model is stored in `model.joblib` and loaded when the API starts. Both Docker images expose the same two endpoints: `GET /healthz` and `POST /predict`. The test message “Congratulations! You won a free prize” was classified as `spam` in both images.

Image	Size (bytes)	Reduction
Naive, <code>python:3.13</code>	502,799,034	–
Multi-stage, <code>python:3.13-slim</code>	132,430,950	73.66%

The size difference is not coming from removing individual Python packages. The naive image keeps the full `python:3.13` runtime as its final filesystem and installs the requirements directly into it. In the multi-stage build, the requirements are installed under `/install` in a builder based on `python:3.13`; the final image starts from `python:3.13-slim` and copies only that installed environment plus `api.py` and `model.joblib`. The builder’s larger base image and its build-time files are therefore not part of the final image. The measured reduction was

$$\frac{502799034 - 132430950}{502799034} \times 100 = 73.66\%.$$

2 Redis cache with Docker Compose

For the second part, Redis was added in front of model inference. The API uses the exact input text as the basis of the Redis key. On a miss, it calls `model.predict()`, stores the label with a 60 s TTL, and returns it. On a hit, the cached label is returned without running the model. The two services are `api` and `cache`; the API reaches Redis at the Compose service name `cache:6379`. The host mapping used for the API was `8001:8000`.

The measured benchmark used 20 miss/hit pairs. A fresh input was sent once, followed immediately by the same input. The recorded medians were:

Metric	Miss	Hit
Median latency (ms)	3.081	1.781
Mean latency (ms)	3.950	1.982
Median speedup	1.73×	

The Compose logs also show the expected pattern of `cache MISS` followed by `cache HIT`, while every request returned HTTP 200. For this workload, the useful part of Compose is that it starts and networks the API and Redis containers as one application on one machine. Kubernetes addresses a different scale: it manages workloads across a cluster and provides controllers for deployment, scheduling, and recovery.

3 Indexed Job for CSV validation

This workload is independent of the spam API. I generated eight deterministic CSV shards with known invalid-row counts. The cluster used for the run had two nodes with two allocatable CPUs each. I used

$$\text{parallelism} = 4$$

because each validator pod requests and limits one CPU. Four pods therefore consume the available four CPUs, allowing four shards to be processed at once; the remaining four completions are scheduled as earlier pods finish.

The actual run showed four pods in `Running` state at the same time. The pod listing also shows the placement across the two nodes:

Shard	Pod	Node
0	shard-validator-0-b8jhg	aiops-m3-m02
1	shard-validator-1-zjpcw	aiops-m3-m02
2	shard-validator-2-b5ljz	aiops-m3
3	shard-validator-3-lmhds	aiops-m3

The job then reported all eight shard results through pod logs: 1, 2, 3, 4, 5, 6, 7, 8 invalid rows for shards 0 through 7 respectively. The logs were read through the Kubernetes Python API rather than written to a shared volume. That avoids depending on the Minikube storage layer for a result that is naturally attached to the pod that produced it; the required output can be collected directly from the Job's pods.

With 3 nodes and 2 CPUs each, the cluster would expose 6 CPUs. With the same one-CPU-per-pod design, I would raise parallelism from 4 to 6 so six shard pods could run together while the remaining two wait for a slot.

4 Deployment, self-healing, and rolling update

The spam API was deployed as a Kubernetes Deployment with two replicas, CPU requests/limits, and an HTTP readiness probe on `/healthz`. The first image returned

```
"status":"ok","version":"v1"
```

and the prediction endpoint returned `"label":"spam"`.

For the self-healing test, one running pod, `spam-api-76d98785bf-9fh2p`, was deleted. Before deletion there were two ready replicas. After deletion, the Deployment again reached two ready replicas, with a new pod `spam-api-76d98785bf-m2g4g`. The Deployment creates and manages a ReplicaSet, and the ReplicaSet controller maintains the requested replica count; when one pod disappears, another is created to restore that count.

For the update test, the only application change was the health response version, from `v1` to `v2`. A new image tag, `aiops-spam-api:k8s-v2`, was built and applied to the Deployment. The rollout completed with revision 2, and an in-cluster health probe sent 120 requests during the update with

```
requests = 120,      failures = 0.
```

The final two pods were both `Running`, and `kubectl rollout history` showed revisions 1 and 2.

A Deployment is intended to keep a service running: it maintains replicas, uses readiness to decide when a new pod can receive traffic, and replaces old pods during a rolling update. The Indexed Job in Question 3 has the opposite lifecycle: it has eight finite completions, each pod handles one shard, and once all completions finish, the workload is done rather than continuously maintained.

AI usage disclosure

AI assistance was used for the shell script that automates build, deployment, testing, and evaluation of the assignment, and for debugging issues encountered while running the code. AI was also used to organise the repository structure and to prepare this write-up from the assignment requirements and the run-time results recorded in the attached evidence. The implementation and final reported measurements were checked by running the repository locally.

Summary of measured results

Q1	502,799,034 B → 132,430,950 B; 73.66% reduction; API checks passed.
Q2	20 benchmark pairs; 3.081 ms miss median vs. 1.781 ms hit median; 1.73× median speedup.
Q3	4 concurrent pods observed; 8/8 shards reported; invalid-row counts 1–8.
Q4	2 replicas; pod recreated after deletion; revision 1 → 2; 120/120 health checks passed during rollout.
